

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

Table of Contents

1. Project Overview	1
2. Background & Business Objective	1
3. Product Purpose	2
4. System Functionalities	2
5. User Stories & Use Cases	3
6. Features Included (Scope Definition)	3
7. Features Not Included (Scope Control)	3
8. Assumptions & Constraints	4
9. Risks & Questions Throughout Development	4

Project Name: Yama-Inventory

Version: 1.0

1. Project Overview

- **Problem Statement:** Inventory managers in manufacturing environments operate in fragmented, manual workflows. Information arrives through disconnected channel such as emails from sales about incoming orders, messages from production about work order changes, supplier confirmations and delays, finance updates on budget and none of it lands in the ERP automatically. The manager is the integration layer: reading every message, manually keying updates into the ERP, then checking the latest state across inventory, sales orders, supplier lead times, and budget before deciding what to do. Once a decision is made, execution is also manual, drafting a purchase order, sending a follow-up email, switching to a backup supplier, modifying a quantity mid-order, every step requires human keystrokes. This produces a three-stage bottleneck: collecting and entering data, analyzing the latest state, and executing the decision. AI recommendation tools only address the middle stage; the manager is still left to do the data wrangling before and the execution after.
- **Target Domain:** Yama-inventory targets to perform automation for discrete manufacturing which cover the full loop from signal ingestion (cross-departmental emails and updates) through ERP synchronization, decision-making, and execution (purchase orders and supplier follow-ups), so that routine procurement runs end-to-end without manual handoffs.
- **Proposed Solution Summary:** Yama-inventory is an autonomous procurement agent that closes the entire fragmented loop. Rather than generating recommendations for a human to act on, Yama-inventory uses Z.AI GLM's native function-calling, multi-step reasoning, and long context window to handle every stage of the workflow: it ingests

inbound signals from other departments (sales, production, finance, suppliers), updates the ERP state directly, reasons across the latest data to decide what to do, and then executes the decision by placing, modifying, cancelling, or switching suppliers on purchase orders, and sending suppliers or other departments follow-up emails. Yama-inventory user interface is deliberately narrow: a Control Center that displays the state of work the agent is doing, and a tuning page where the manager refines the values and rules the agent operates by.

2. Background & Business Objective

- **Background of the Problem:** Traditional inventory management follows a manual, siloed workflow. A sales order arrives by email and is manually keyed into a spreadsheet. Stock levels are checked separately in a warehouse system. The production team updates work order status in another tool. Finance holds budget data in yet another system. Suppliers send confirmations, delay notices, and quote updates through email threads that no one consolidates. No single view connects all these signals and even when the data is eventually entered, no system acts on it.

The inventory manager becomes the integration layer between these silos. A typical day involves reading dozens of emails from sales, production, finance, and suppliers, then manually transcribing the relevant updates into the ERP so that the system reflects current reality. Only after this data wrangling can the manager analyze the latest state, checking inventory levels against open sales orders, cross-referencing work order schedules, validating budget headroom before deciding what procurement action is needed and how urgent is it. And once the decision is made, execution is yet another round of manual work: drafting a purchase order, sending it to the supplier, following up when the supplier does not confirm, modifying the order if production needs change, or switching to a backup supplier when the primary stocks out. Every one of these steps requires human keystrokes.

This three-stage bottleneck (collect, analyze, execute) is where the day disappears. The work itself is rarely intellectually demanding; most of it is routine, pattern-following, and well within the bounds of established company practice. But because each stage requires a human in the loop, the manager's calendar fills up with low-novelty execution work and leaves little time for the genuinely novel decisions that require human judgment. AI recommendation tools have so far only addressed the middle stage, and even there only partially: they can analyze data once it is in the system, but they cannot ingest the upstream signals, cannot update the ERP, and cannot execute the downstream actions. The manager is still doing the boring two-thirds of the work, plus reviewing whatever the AI suggests for the middle third. The fragmentation persists because automation has been pointed at the analysis part only.

- **Importance of Solving This Issue:** Solving the fragmented, three-stage workflow problem is critical because it directly impacts a manufacturer's bottom line, operational agility, and the quality of human attention available for the decisions that matter. Yama-inventory delivers value in four specific areas:
 1. **Revenue Protection & Maximization:** Slow procurement directly costs revenue. When low-stock signals sit in an inbox for hours before the manager has time to draft a PO, lead times compress and stockouts become more likely and every stockout is either a lost sale or a delayed customer order with penalty exposure. When supplier follow-ups get forgotten on a busy day, late deliveries cascade into halted production lines, causing big downtime cost. When backup-supplier switches happen too late because the manager only noticed the primary's stockout the next morning, the choice narrows to expensive expedited shipping or missed delivery commitments.
 2. **Drastic Reduction in Time-to-Decision:** In modern supply chains, the gap between a signal arriving and an action being taken is where small problems become

large ones. When this gap depends on whether the manager has read their inbox, updated the ERP, and found time to act, response time is measured in hours or days. With Yama-inventory ingesting signals, updating state, and executing routine actions autonomously, that gap shrinks to seconds for the routine cases.

3. Reclaiming Manager Bandwidth for High-Value Work: The bulk of an inventory manager's day is consumed by routine execution: reading emails, keying updates into the ERP, drafting standard purchase orders, sending follow-ups, and switching to backup suppliers when the primary stocks out. None of this work demands judgment, but all of it demands human keystrokes. By automating the full collect-analyze-execute loop for routine actions, Yama-inventory frees the manager to focus on the genuinely novel decisions — supplier relationship building, contract negotiation, strategic sourcing, and the high-stakes exception cases that the agent escalates rather than handles silently. The shift is from per-action approver to values author and exception handler.

4. Auditable, Defensible AI-Driven Procurement: Autonomous procurement is only acceptable if every action is fully traceable. Yama-inventory logs every autonomous action with its trigger signal, the preferences applied, the alternatives considered and rejected, the confidence score, the guardrails it cleared, and the manager's overrides if any. Without this audit trail, no manufacturer would let an agent send a real PO; with it, autonomy becomes a strategic capability rather than a liability.

- **Strategic Fit / Impact:** Yama-inventory aligns with the Z.AI GLM ecosystem and the broader industry shift "AI as advisor" to "AI as bounded delegate", agentic systems that move beyond answering questions or generating recommendations toward taking sequential, reasoned actions across multiple tools and being held accountable for the outcomes. The current generation of AI procurement tools stops at the recommendation layer because trusting an LLM with real authority feels reckless when its reasoning is opaque and its behavior is generic. Yama-inventory strategic fit is the architectural pattern that makes autonomy viable in industrial operations: pair GLM's reasoning with

an explicit, manager-defined values profile so the agent's decisions reflect the company's actual priorities rather than a default and constrain GLM's authority with a deterministic guardrail layer so the agent cannot violate hard rules even when its reasoning suggests otherwise. The combination of GLM for judgment and deterministic code for limits is what turns "AI in the loop" into "AI runs the loop, with human oversight."

Yama-inventory impact: By closing the full collect-analyze-execute workflow rather than just the analyze stage, Yama-inventory shows that agentic AI can hold real, consequential authority in manufacturing operations while remaining values-aligned, auditable, and reversible. The result is a system that does not just speed up procurement, it also changes the manager's role from per-action approver to values author and exception handler, and proves that agentic AI delivers concrete, measurable business value at the operational layer of an enterprise rather than just the conversational surface.

3. Product Purpose

3.1. Main Goal of the System: Yama-inventory aims to provide an autonomous procurement agent that closes the full collect-analyze-execute loop in manufacturing inventory management. Rather than connecting data silos so a human can act faster on the result, Yama-inventory acts directly: ingesting signals from cross-departmental emails and updates, synchronizing the ERP with the latest state, reasoning across the data through Z.AI GLM, and executing routine procurement actions like purchase orders, modifications, cancellations, supplier switches, and follow-ups within authority defined by a manager-curated Company Values & Decision Profile. The core objective is to remove the manager from the routine execution path entirely, while ensuring that every autonomous action reflects the company's documented priorities, and that genuinely uncertain or high-stakes situations are escalated with full context for human decision. Ultimately, Yama-inventory shifts the human role from manually collecting data, analyzing state, and executing decisions to authoring the values the agent operates by, reviewing the audit trail of work completed, and handling the exception cases the agent flags.

3.2. Intended Users (Target Audience):

- Inventory Manager / Supply Chain Planner: primary operator. Reviews the Control Center to track autonomous actions in progress, handles escalations the agent flags, and overrides individual decisions when needed.
- Operations Manager: defines and updates the Company Values & Decision Profile that grounds the agent's behavior. Reviews proposals to promote repeated overrides into permanent rules.
- Procurement Officer: secondary user. Maintains the supplier whitelist, price thresholds, and approved-supplier metadata that feed the guardrail layer.
- Supply Chain Analyst: audits historical action records, exports reasoning traces for compliance reviews and post-mortems, and identifies patterns in agent behavior or override frequency.

4. System Functionalities

4.1. Description: Yama-inventory operates as an autonomous procurement agent built on the Z.AI GLM ecosystem. Unlike systems that wait for a manager to trigger an analysis, Yama-inventory runs continuously: it monitors inbound signals from cross-departmental channels (sales emails confirming new orders, production messages updating work order schedules, finance updates on budget changes, supplier communications about confirmations and delays), parses these unstructured signals into structured updates, and synchronizes the ERP state automatically. When a signal warrants action like a stock level crossing reorder threshold, a late supplier confirmation, a demand forecast shift, a production schedule change and the agent reasons across the latest state through GLM, consults the manager-defined Company Values & Decision Profile, and produces a candidate action with full reasoning, alternatives considered, and confidence score. The candidate passes through a deterministic guardrail layer that enforces hard rules (budget caps, supplier whitelist, value ceilings, anomaly checks) and is routed onto a four-level autonomy gradient based on risk and confidence. Routine, in-policy, high-confidence actions are executed directly, the agent sends purchase orders, modifies or cancels open POs, switches suppliers, and dispatches follow-up emails. Higher-risk or low-confidence actions are

escalated to the manager with a complete decision packet. Every action is logged in the Control Center with its full reasoning trace, and manager overrides feed back into the values profile to refine future behavior.

4.2. Key Functionalities:

- **Continuous Signal Ingestion and ERP Synchronization:** The agent monitors inbound channels (cross-departmental emails, supplier messages, system alerts) and parses unstructured updates into structured ERP writes. The manager no longer keys data into the system manually, GLM extracts the relevant fields from each message, validates them against existing state, and updates the mock ERP (CSV layer) directly. Signals that cannot be parsed with sufficient confidence are routed to a triage queue rather than silently dropped.
- **Two-Action Autonomous Execution:** The agent's authority is constrained to two action types covering the bulk of routine procurement work:
 - **Order Action:** create new PO, modify existing PO (quantity or delivery date), cancel PO, switch supplier on an open PO.
 - **Follow-up Action:** auto-generated supplier or cross-departmental outreach for late confirmations, late deliveries, missing documents, or scheduled status checks.
- **Company Values & Decision Profile (Manager-Defined):** A two-channel interface where the manager grounds the agent's behavior. Deterministic policies (budget caps, supplier whitelist/blacklist, single-PO ceilings, mandatory-quote thresholds) feed the guardrail layer as enforceable code. Contextual values (free-text statement of priorities, 2-3 example past decisions with reasoning) feed the GLM context window for grounding. This separation keeps non-negotiable limits enforceable while allowing nuanced preferences to shape behavior within those limits.
- **Bounded Autonomy Gradient:** Each candidate action is routed to one of four execution levels based on value, novelty, and confidence: Auto-Execute (committed silently), Auto-Execute + Notify (committed with manager digest),

Propose & Wait (manager confirmation required), and Escalate (out-of-policy, low-confidence, or anomalous - routed immediately with structured options). This avoids the binary trap of "always ask" versus "always act."

- **Multi-Layer Guardrail Engine:** A deterministic policy gate sits between GLM and any tool call. GLM proposes; guardrails dispose. The gate enforces budget caps, supplier whitelist/blacklist, single-PO value ceilings, anomaly detection (orders > 1.5x typical SKU size, suppliers with no prior history, sudden price changes), and rate limits. Repeated violations on similar signals are flagged as possible policy gaps.
- **Soft-Send Buffer:** Auto-executed Order actions enter a 15-minute cancellable window before transmission to the supplier. Manager receives an in-app notification and can cancel from the Control Center with one click - bounding the cost of any single autonomous mistake.
- **Control Center:** The primary manager-facing surface. Displays the Action Stream (chronological feed of all autonomous actions: executed, pending soft-send, awaiting manager response, escalated, blocked), Action Detail (trigger signal, values applied, alternatives considered and rejected with reasons, confidence score, guardrails cleared, expected KPI impact, current status), and Override & Feedback (cancel a soft-send, override a completed action, attach a written correction).
- **Override-as-Learning Loop:** When a manager overrides an action, the override and reason are captured as a new few-shot example appended to the contextual-values channel. After three similar overrides on related signals, the system proposes a new explicit deterministic rule for the manager to confirm - promoting fuzzy preference into hard policy as company values evolve.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: Yama-inventory utilizes the Z.AI GLM (General Language Model) ecosystem as its core orchestration engine. This model was explicitly chosen over

standard conversational chatbots due to its superior capabilities in enterprise-grade, agentic workflows:

- **Native Function Calling:** GLM reliably invokes external tools and formats arguments precisely, which is essential when the agent is autonomously writing to the ERP, querying supplier history, and dispatching purchase orders.
- **Multi-Step Reasoning:** GLM holds complex cross-functional logic across sequential tool invocations; ingesting an email signal, querying inventory state, checking open POs, consulting supplier history, and arriving at a candidate action with alternatives considered without losing context across the chain.
- **Long Context Window:** GLM can hold the Company Values & Decision Profile (hard rules, contextual preferences, few-shot examples) and the current ERP state across multiple data sources.

4.3.2. Prompting Strategy: Yama-inventory uses a three-layer multi-step agentic prompting strategy, structured by how frequently each layer changes:

System Layer (static): Defines the agent's role, action vocabulary (Order and Follow-up with their sub-types), output schema for candidate actions, and guardrail-aware behavior; the agent is instructed that it may propose any action but that the guardrail layer validates before execution.

Values Layer (infrequently updated, cached): Contains the structured policy summary, the manager's free-text values statement, and 2–3 few-shot example past decisions with reasoning drawn from the Company Values & Decision Profile. Aggressively cached because this layer rarely changes between actions, caching here directly reduces per-action token cost.

Task Layer (per-action): The trigger signal and tool outputs collected progressively during the agent's reasoning loop. Rather than dumping all available data upfront, the agent issues sequential targeted tool calls, for example, an inbound email triggers `get_inventory` for the

relevant SKU, which informs `get_supplier_history` for the shortlisted suppliers, which informs `get_open_pos` to check existing commitments, which informs `get_finance_state` to validate budget headroom. Each call is small and targeted, keeping token usage bounded and reducing hallucination risk from irrelevant context.

4.3.3. Context & Input Handling: The system operates proactively by continuously monitoring the simulated enterprise environment. When a disruption is detected (e.g., a rush order or an inbound supplier shortage email), the `DataRepository` class immediately parses the current state of the local CSV files to construct an up-to-the-minute reality of inventory, active BOMs, and schedules. Crucially, the manager's "Company DNA", comprising custom KPI weightings, customer priority tiers, and strict financial guardrails configured in the Preferences page, is dynamically injected directly into the AI's system prompt as absolute operating laws. To manage token limits effectively and prevent context dilution, the backend server pre-filters the CSV data, transmitting only the specific SKUs, active work orders, and dependencies relevant to the immediate disruption rather than the entire database.

4.3.4. Fallback & Failure Behavior: Given the high financial stakes of autonomous supply chain execution, the system is designed to fail safely and transparently. System-level errors, such as WebSocket timeouts or malformed JSON outputs from the LLM, immediately halt the orchestration process and log a designated error card in the Command Centre. There are no blind automatic retries; the system prioritizes data integrity.

Most importantly, the traditional "human-in-the-loop" bottleneck has been replaced by a strict Guardrail Exception Protocol. The AI executes reallocations and sends Purchase Orders entirely autonomously unless a calculated action breaches a user-defined threshold (e.g., an emergency air-freight PO exceeding RM 5,000, or dropping below a minimum safety stock). If a guardrail is hit, autonomous execution halts, the system queues the drafted actions, and it generates an "Approval Required" flag in the Command Centre for human intervention. Furthermore, execution is treated as an atomic operation: if the AI successfully updates the CSV

inventory ledger but encounters an error sending the automated email to the supplier, the system automatically rolls back the CSV changes to prevent "phantom" inventory states.

5. User Stories & Use Cases

- **User Stories:**

- *The "Set and Forget" Setup: "As an Inventory Manager, I want to define my company's business priorities (KPIs) and financial guardrails so that the AI agent understands how we do business and can make autonomous decisions aligned with our goals."*
- *The Autonomous Execution: "As an Operations Director, I want the AI to autonomously intercept supply chain disruptions (like rushed VIP orders or supplier delays), calculate the mathematically optimal reallocation, and automatically issue the necessary Purchase Orders, so that I don't have to manually act as a bottleneck."*
- *The Transparent Audit: "As a Supply Chain Executive, I want to log into a streamlined Control Center to see a clear audit trail of all actions the AI took while I was offline, including its reasoning tied directly to our company KPIs, so that I can trust the system is working correctly."*

- **Use Cases (Main Interactions):**

- **Use Case 1: Programming the "Company DNA" & Guardrails**

- **Flow:** The user navigates to the Preferences interface to set the rules of engagement. They use sliders to weigh competing KPIs (e.g., 70% Priority on "Fulfilling VIP Orders", 30% on "Minimizing Freight Costs"). They establish strict guardrails, such as "Auto-approve POs under RM 5,000" and "Never drop raw material safety stock below 10 units."
- **Result:** The Agent's system prompt is dynamically updated to lock in these absolute laws, dictating how it will score future automated decisions.

- **Use Case 2: Autonomous Orchestration & Audit Logging (The "Happy Path")**
 - **Flow:** A background trigger occurs (e.g., a VIP customer submits an urgent rush order on April 21 that conflicts with an existing work order). The AI intercepts the event, cross-references live ERP data, and simulates outcomes.
 - **Action:** Finding a solution that fits the Company DNA, the AI autonomously reallocates the inventory to the VIP, drafts a replenishment PO to a backup supplier, and sends the PO directly via email/API without human intervention.
 - **Result (Control Center):** When the manager checks the UI, the inventory panels are gone. Instead, they see a clean Audit Card: Action Executed: Reallocated 120 units to SO-VIP. Sent PO #8892 for RM 1,200. The card includes a "Reasoning" dropdown explaining how this action maximized the VIP KPI while staying under the RM 5,000 auto-spend guardrail.
- **Use Case 3: Exception Handling (Human-in-the-Loop for Guardrail Breaches)**
 - **Flow:** A massive supplier delay requires an emergency air-freight restock that costs RM 8,500. The AI calculates this is the only way to save a critical order, but it hits the RM 5,000 financial guardrail set in Use Case 1.
 - **Action:** The AI halts autonomous execution. It drafts the email and the PO but sends an urgent notification to the manager's Control Center.
 - **Result:** The manager sees a pending action card: APPROVAL REQUIRED: Emergency PO exceeds RM 5,000 limit. The manager reads the AI's justification, clicks "Approve," and the AI immediately fires off the queued communications.

6. Features Included (Scope Definition)

- **"Company DNA" Configuration Interface:** A control panel for managers to set business preferences, KPI weightings (e.g., speed vs. cost), and strict financial guardrails to govern the AI's logic. Point-in-time filtering: every data source versioned with valid_from timestamps.
- **Autonomous AI Orchestration:** The agent automatically cross-references data, resolves inventory conflicts, and executes decisions without human intervention, provided the actions fall within the predefined guardrails.
- **Executive Control Center (Audit Log):** Replaces the traditional raw inventory dashboard with a streamlined feed of executed actions, displaying the status of work completed and the AI's KPI-aligned reasoning for each decision.
- **Automated Email Execution:** The AI automatically drafts and sends follow-up communications to other departments and Purchase Orders (POs) directly to suppliers via email to resolve shortages.
- **CSV-Based Data Modeling:** The system utilizes CSV files to act as the primary database, simulating all internal company ERP data, inventory levels, BOMs, and sales orders.

7. Features Not Included (Scope Control)

- **Historical Email Threading & Context Retention:** The system treats every inbound email as an independent, stateless event. It does not map dependencies between old and new emails, nor does it include complex lifecycle-deletion algorithms to manage long-term email memory.
- **Spam Filtering & Phishing Detection:** The prototype assumes a trusted sender environment. It does not include security mechanisms to filter out spam, authenticate sender identities, or detect malicious actors; all ingested emails are assumed to be legitimate supply chain communications.

- **Live ERP or WMS API Integration:** No direct API connections to commercial systems like SAP, Oracle, or NetSuite. All enterprise data is strictly modeled via the provided CSV architecture.
- **Third-Party Messaging APIs:** No integration with external chat applications (e.g., WhatsApp, Slack, SMS). Email acts as the single simulated communication channel for the agent.
- **Human-in-the-Loop for Every Action:** Manual human approval is not required for standard execution; it is only triggered as an exception when the AI detects a financial guardrail breach.
- **Multi-user Collaboration:** No role-based access control (RBAC) or multiple concurrent user accounts.
- **Mobile Application:** The interface is designed for desktop/web use only.
- **Multi-Language Support:** English only.

8. Assumptions & Constraints

- **Universal Enterprise Modeling (CSV & Email Assumption):** We assume that every target company utilizes a completely different, often highly customized mix of legacy ERPs (SAP, Oracle, NetSuite) and communication channels. To make our hackathon prototype universally applicable without relying on proprietary API access, we use local CSV files to model the structured ERP database and automated Email generation to model the unstructured external communication channels.
- **Autonomous Execution & Guardrail Constraints:** We assume the system is trusted to act autonomously based on pre-set "Company DNA." The agent will automatically reallocate inventory and issue external Purchase Orders without human review. The constraint is that it operates strictly within user-defined financial and confidence

guardrails (e.g., auto-spend limits). Human intervention is only required if a proposed action breaches these hard constraints.

- **LLM Cost Constraint:** Estimated token usage per orchestration session is approximately 8,000–15,000 tokens (input + output across all tool calls). At current LLM pricing, this is roughly \$0.024–\$0.045 per session. To keep costs manageable, tool responses are pre-filtered server-side to only the relevant time window, passing the full unfiltered CSV to the model on every call would increase input tokens by 3–4x. Prompt caching is also applicable for repeated tool schemas.
- **Technical Constraints:** The system currently runs locally (FastAPI + React on localhost). No cloud deployment, production database (SQL/NoSQL), or authentication layer is included in this prototype scope.
- **Performance Constraints:** End-to-end agentic orchestration time is roughly 15–45 seconds depending on LLM API response latency, the complexity of the multi-variate math, and the number of tool calls required. This latency is communicated to the user via streaming status messages in the Control Center. No generation limit is enforced per session.
- **Stateless Communication (No Historical Dependency):** We assume that every incoming email represents an independent, self-contained event. New emails do not depend on the context of older emails to be understood or acted upon. Because of this assumption, the system processes communications statelessly—once an email triggers an action and the database is updated, the email's historical context is no longer required for future LLM reasoning. This eliminates the need to engineer complex context-retention or lifecycle-deletion algorithms for the prototype.
- **Trusted Sender Environment:** Due to the strict time constraints of the hackathon, we assume that all ingested emails originate from verified, legitimate actors (e.g., actual suppliers, internal personnel, or trusted clients). The system implicitly trusts the sender's

identity and the data payload. Implementing robust zero-trust email security, phishing detection, or spam filtering is outside the scope of this proof-of-concept.

9. Risks & Questions Throughout Development

- **Data Freshness & State Synchronization:** How do we ensure the agent's internal state remains perfectly synchronized with the real-world warehouse in a live deployment? While our current CSV architecture effectively models the environment for this prototype, a production deployment relies on live API syncs. If an external ERP webhook drops or is delayed, there is a risk the AI could autonomously reallocate inventory based on stale data, leading to physical stock discrepancies on the warehouse floor.
- **Autonomous Execution Hallucination:** What happens if the LLM misinterprets the "Company DNA" and autonomously issues a Purchase Order that exceeds the financial budget, or steals stock from a VIP? Because the system now executes rather than just recommends, hallucination is financially material. We mitigate this using strict structured output schemas (JSON) and hardcoded server-side validation that intercepts the action, blocks the automated email, and forces a human-in-the-loop exception if guardrails are breached.
- **Command Centre Audit Accuracy & "Silent Failures":** Since the Command Centre is the manager's sole window into the AI's autonomous actions, how do we guarantee absolute consistency between the UI audit log, the backend CSV data mutations, and the external emails sent? There is a risk of "partial execution" (e.g., the AI updates the inventory CSV and logs a "Success" in the Command Centre, but the actual PO email fails to send due to an external network error). Ensuring atomic operations so that a failure in one step safely rolls back the entire action is critical for user trust.